PROJECT DOCUMENTATION

Data Engineering PoC - Olist E-Commerce Data Warehouse

End-to-End Data Engineering Pipeline using Medallion Architecture

Prepared by	Internship / Organization	Completion Date
Sudharsan S A	KANINI	19 / 07 / 2026

PostgreSQL | Docker | Apache Airflow | FastAPI | React | Great Expectations

Documentation Revision 2.0 — reflects the current implemented state of the platform.

1. Executive Summary

This project implements an end-to-end data engineering proof of concept for the Olist e-commerce dataset. It ingests CSV source data into PostgreSQL, applies a complete Bronze-Silver-Gold Medallion Architecture, validates warehouse outputs with Great Expectations, exposes analytics through a FastAPI backend, and presents the results in a fully responsive React business intelligence dashboard.

The platform has since matured beyond the original proof of concept. It now delivers a complete Medallion Architecture, a responsive React dashboard, a dedicated Business Intelligence module, a Data Quality Center, Great Expectations validation, a metadata and observability framework, a FastAPI backend, Apache Airflow orchestration, SCD Type 1 and Type 2 customer history, batch control, audit logging, interactive analytics, and executive reporting.

The solution is designed to demonstrate practical warehouse engineering: raw data traceability, standardized Silver datasets, Gold analytics tables, batch metadata, incremental loading, automated validation, Apache Airflow orchestration, and customer Slowly Changing Dimension (SCD) processing, all surfaced through a governed API and an executive-grade dashboard.

Objective	Delivered capability
Reliable data pipeline	CSV ingestion, batch status tracking, and Docker-hosted PostgreSQL.
Analytics-ready warehouse	Gold revenue, product, and seller performance tables.
Operational governance	Metadata control, validation reporting, incremental customer loading, and SCD history.
Data quality assurance	Great Expectations validation suites with a quality score, expectation results, and dashboard visualization.
Observability	Batch control, audit logging, pipeline health, and warehouse monitoring surfaced through dedicated APIs.
Business consumption	FastAPI endpoints and a responsive, multi-page React business intelligence dashboard.

2. Project Architecture

The architecture separates ingestion, conformance, analytics, validation, and presentation responsibilities. PostgreSQL stores the warehouse schemas; Docker provides the database runtime; Airflow coordinates the warehouse stages; Great Expectations validates warehouse outputs; FastAPI serves governed API responses; and React consumes those responses for a responsive dashboard.

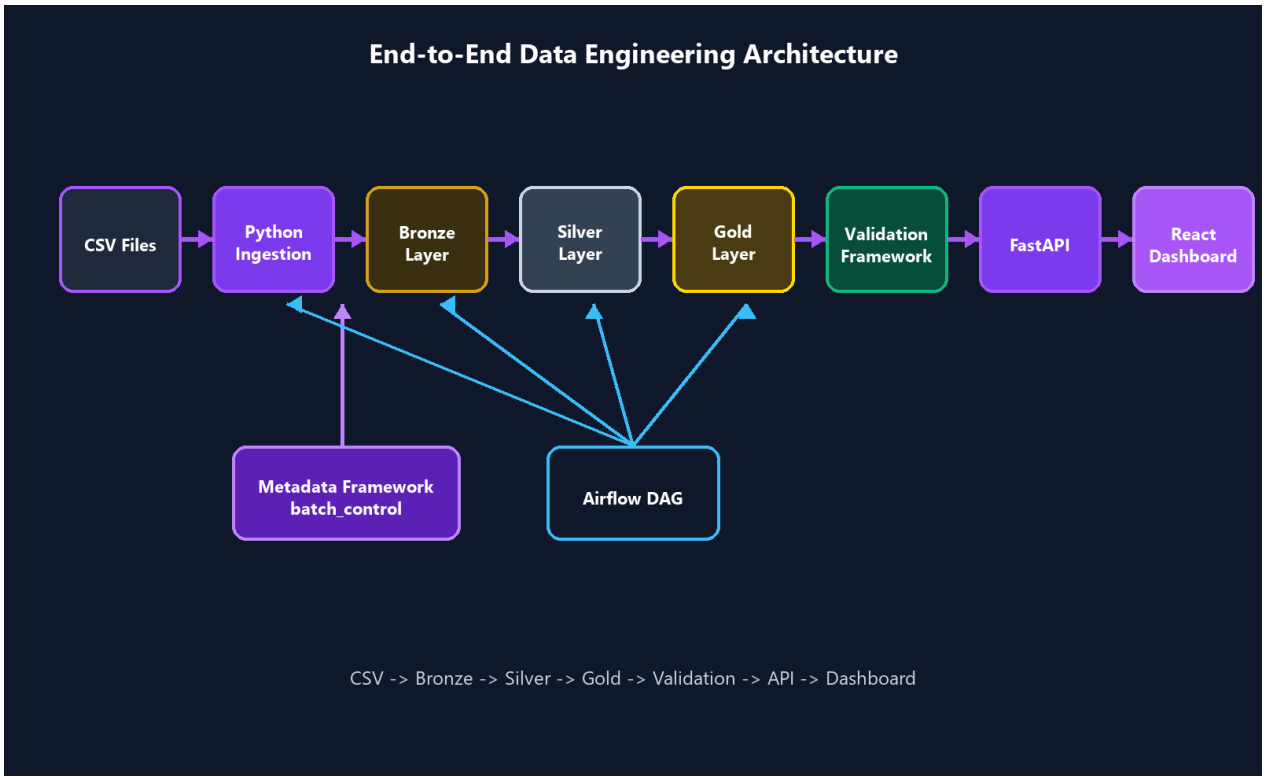


Figure 1. Supplied end-to-end architecture diagram: ingestion, metadata, orchestration, validation, API, and dashboard flow.

Component	Role in the solution
Docker + PostgreSQL	Runs the de_poc database in a reproducible containerized environment.
Bronze / Silver / Gold	Implements raw capture, conformance, and analytics aggregation layers.
Apache Airflow	Runs ingestion, Silver transformations, Gold transformations, and validation as a DAG.
Great Expectations	Executes expectation suites against warehouse tables and persists validation history and quality scores.
FastAPI	Provides overview, pipeline, quality, medallion, analytics, and SCD endpoints behind a consistent response envelope.
React	Renders the Executive, Revenue, Customer, Seller, Category, Geographic, Data Quality, Dataset Explorer, and Pipeline Health views.

3. Technology Stack

Technology	Purpose
Python	Pipeline scripts, ingestion utilities, validation, SCD processing, and FastAPI services.
PostgreSQL	Warehouse database (de_poc) with bronze, silver, gold, and metadata schemas.
Docker	Reproducible PostgreSQL 16 runtime and persistent postgres_data volume.
SQLAlchemy	Database engine and transactional batch control during CSV ingestion.
Pandas	Reads source CSV files and loads raw Bronze tables.
Great Expectations	Declarative validation suites for null, uniqueness, referential integrity, volume, and positive-value checks, with persisted quality scoring.
FastAPI	REST API layer with a consistent data/meta response envelope.
React + TypeScript	Type-safe dashboard frontend rendering the Business Intelligence and observability views.

Technology	Purpose
React Query	Client-side data fetching, caching, and loading/error state management for every dashboard page.
Recharts	Revenue trends, category rankings, funnels, and treemap visualizations.
Tailwind CSS	Utility-first styling for the responsive, adaptive dashboard layout.
Vite	Frontend build tool and development server for the React application.
Apache Airflow	Orchestrates the warehouse_pipeline DAG.

4. Multiple Dataset Ingestion

The Olist e-commerce dataset represents marketplace activity across customers, orders, products, sellers, payments, reviews, and geolocation. Each source dataset is ingested independently through its own ingestion process, while lineage across datasets is preserved through shared batch and source metadata.

The following datasets are ingested independently: customers, orders, order_items, payments, products, sellers, reviews, geolocation, and category_translation. Each ingestion process reads its own CSV extract, creates a metadata.batch_control record, and appends rows to its corresponding Bronze table without depending on the completion of any other dataset's ingestion, while batch_id and source_system values tie every dataset back to a common lineage trail.

Source file / entity	Warehouse use
customers	Customer identity and city/state attributes; source for customer SCD processing.
orders	Order lifecycle timestamps, status, and customer relationship.
order_items	Item-level price, freight, product, and seller records.
payments	Payment type, installment, and payment value.
products	Product catalog and category attributes.
sellers	Seller identity and location data.
reviews	Customer review score and comments for future experience analysis.
geolocation	State, city, latitude, longitude, and postal-prefix reference data.
category_translation	Maps Portuguese product category names to English for reporting.

5. Medallion Architecture

Bronze Layer

Bronze preserves raw ingested records, including batch_id, source_system, and load timestamps. Representative tables include bronze.customers_raw, bronze.orders_raw, bronze.order_items_raw, bronze.payments_raw, bronze.products_raw, bronze.sellers_raw, bronze.reviews_raw, bronze.geolocation_raw, and bronze.category_translation_raw.

Silver Layer

Silver standardizes and cleans operational data. Core tables are silver.customers, silver.orders, silver.products, silver.sellers, silver.payments, and silver.order_fact. Customer city and state are normalized to uppercase, and order_fact joins order headers with item records. silver.customers_scd adds Type 1 and Type 2 customer history support.

Gold Layer

Gold exposes business-ready aggregations: gold.sales_summary, gold.product_performance, and gold.seller_performance. These tables support revenue trends, category rankings, and seller leaderboards in the dashboard, and feed directly into the Great Expectations validation suites.

Raw CSV	→	bronze.*_raw	→	silver.*	→	gold.*

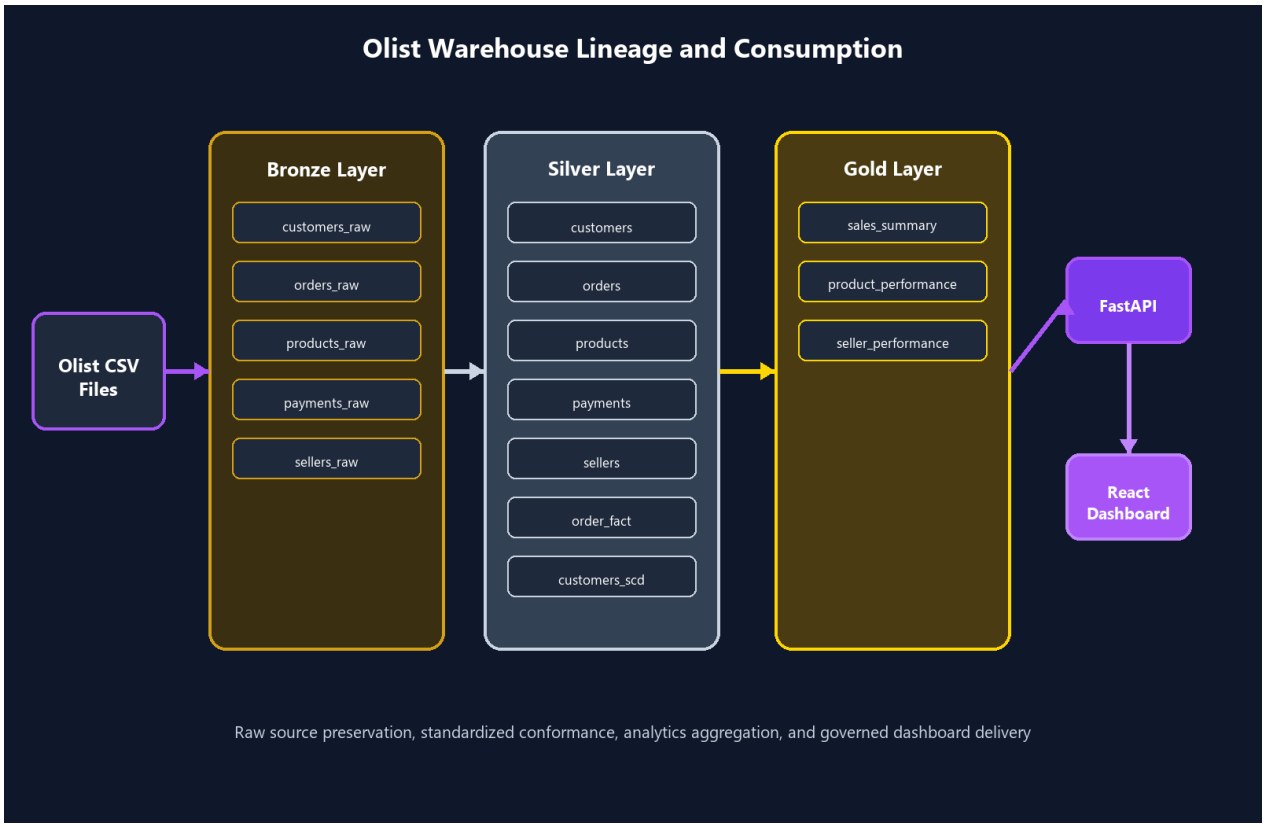


Figure 2. Supplied warehouse lineage diagram: Olist source files through Bronze, Silver, Gold, FastAPI, and the React dashboard.

6. ETL Pipeline

The pipeline follows CSV -> Python ingestion -> Bronze -> Silver -> Gold -> Validation -> API -> Dashboard. Ingestion scripts create a metadata.batch_control record, read CSV data through Pandas, enrich it with batch_id, source_system, and load timestamps, append it to the Bronze schema, and update the batch status when complete.

CSV	→	Ingestion	→	Bronze	→	Silver	→	Gold	→	Validation	→	API

Every stage of the pipeline writes to a shared metadata surface so that lineage, freshness, and outcomes can be reconstructed for any batch:

- batch_id and source_system tag every Bronze, Silver, and Gold record back to its originating load.
- Load timestamps record when each row entered the warehouse.
- metadata.batch_control persists pipeline_name, status, timing, records processed, and records rejected.
- metadata.load_tracker records the last processed batch per dataset to support incremental loading.

- Observability metadata and audit information are exposed through the pipeline and quality APIs and rendered on the Pipeline Health and Data Quality Center dashboard pages.

Capability	Implementation in repository
Batch processing	metadata.batch_control stores pipeline_name, status, timing, processed records, and rejected records.
Incremental loading	scripts/incremental_load.py reads metadata.load_tracker, selects Bronze customer rows newer than last_batch_id, then updates the tracker.
Silver transformation	scripts/silver_transformations.py executes 01_customers.sql through 06_order_fact.sql.
Gold transformation	scripts/gold_transformations.py executes sales, product, and seller aggregation SQL.
Audit tracking	Pipeline and validation endpoints expose batch metadata, records processed, duration, and validation status.

7. Data Quality and Validation

Data quality is enforced through a complete Great Expectations implementation that validates the conformed and analytics layers after every pipeline run. The dashboard now visualizes validation status directly, rather than surfacing quality only through scripts and logs.

Validation category	Purpose
Null validation	Confirms that required fields are populated across Silver and Gold datasets.
Uniqueness validation	Confirms primary identifiers such as customer_id and order_id are not duplicated.
Referential integrity	Checks that order, payment, and item records resolve to valid customer, product, and seller identifiers.
Volume validation	Confirms that expected Silver and Gold datasets contain rows after transformation.
Positive value validation	Confirms price, freight, and payment values fall within expected non-negative ranges.
Data reconciliation	Compares Bronze and Silver entity row counts for customer, order, product, seller, and payment entities.

Every Great Expectations run produces a quality score, a validation history entry, and a set of expectation results. These are exposed through API-driven quality reporting and rendered in the Data Quality Center, including validation success rate, passed and failed expectation counts, total coverage, a validation health trend, and a failure distribution by expectation type.

Data Quality Score = 100 when the latest batch completes successfully with zero rejected records and all expectations pass, as exposed through the quality and analytics dashboard posture.

8. Slowly Changing Dimensions

The project adds silver.customers_scd for customer city/state dimension management. It stores customer_id, city, state, effective start/end dates, is_current, version_number, and created_at. A partial unique index ensures only one current version per customer.

SCD approach	Behavior
SCD Type 1	Overwrites the current customer_city and customer_state values; no additional history version is created.

SCD approach	Behavior
SCD Type 2	Detects city/state changes, expires the current row, records effective_end_date, sets is_current to false, and inserts the next version.

Demonstrated customer: 00012a2ce6f8dcda20d059ce98491703

Version 1	→	Change Detected	→	Version 2

Version	City	Current state	History treatment
1	OSASCO	Historical	Preserved with effective_end_date and is_current = false.
2	DEMO_CITY_A	Current	Inserted as the active row with a new effective_start_date.

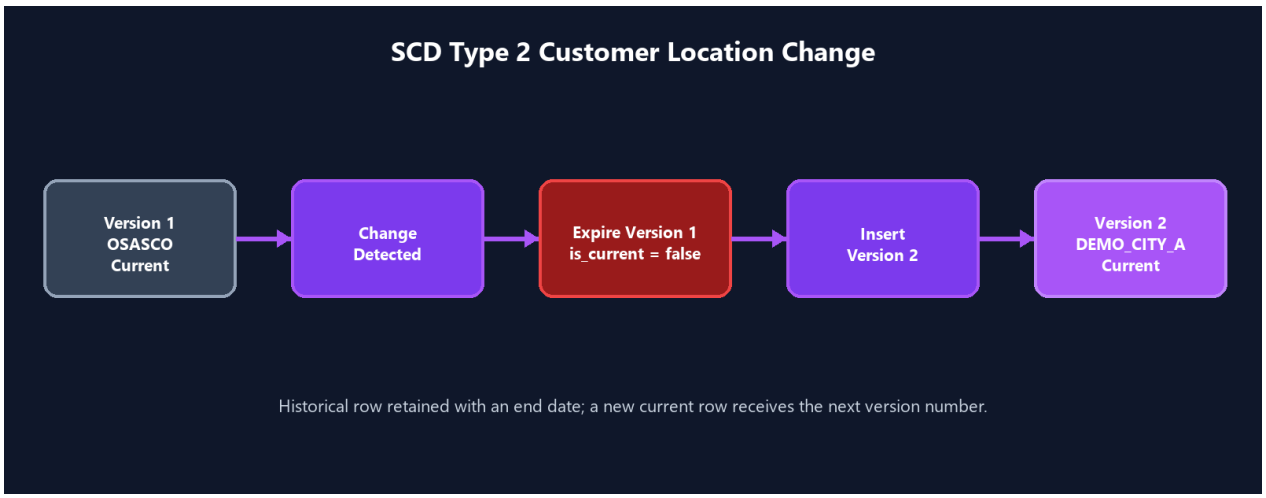


Figure 3. Supplied SCD Type 2 diagram showing the OSASCO to DEMO_CITY_A customer location change.

The FastAPI SCD endpoints are GET /api/v1/scd/summary and GET /api/v1/scd/history/{customer_id}. The dashboard uses the history response to render the version timeline.

9. Airflow Orchestration

airflow/dags/warehouse_pipeline.py defines the warehouse_pipeline DAG with five BashOperator tasks: bronze_ingestion, silver_transformations, gold_transformations, validation, and gx_validation. The DAG is manual (schedule=None), starts on 2026-07-01, and executes the stages sequentially.

bronze_ingestion	→	silver_transformations	→	gold_transformations	→	validation	→	gx_validation

Airflow provides a single operational view for pipeline execution, task status, and failure monitoring, while the batch_control table provides the persisted warehouse metadata that the Pipeline Health page renders as a DAG monitoring view and execution timeline.

10. Audit Framework

Every pipeline run is recorded in metadata.batch_control, giving the platform a complete, queryable audit trail of warehouse activity. Each batch record captures the pipeline execution metadata needed to answer what ran, when it ran, and whether it succeeded.

Audit field	Description
Pipeline execution metadata	pipeline_name, batch_id, and status for every ingestion, transformation, and validation run.
Records processed	Row count successfully written by the batch.
Records rejected	Row count that failed validation or transformation rules within the batch.
Batch start / batch end	Timestamps marking the duration boundary of the batch.
Duration	Elapsed execution time computed from batch start and end.
Pipeline status	RUNNING, SUCCESS, or FAILED state for the batch.
Observability metrics	Throughput and runtime signals derived from batch metadata.
Quality score	Great Expectations success rate associated with the batch's validation run.

Audit information is surfaced on the Pipeline Health dashboard page as records processed, runtime, throughput, pipeline success rate, a pipeline execution timeline per stage, and a DAG monitoring view that reflects the current warehouse dependency sequence and latest validation signal.

11. Business Intelligence Dashboard

The dashboard has been heavily expanded from the original Overview/Dataset Explorer/Pipeline Health/Data Quality set into a full, responsive Business Intelligence suite. Every page is backed by real API data served through the FastAPI response envelope and consumed with React Query.

Executive Dashboard

The Executive Dashboard is the landing view of the platform. It presents executive KPI cards for total revenue, total orders, total customers, and total sellers, alongside a revenue trend chart with day, week, and month granularity, a Deep Dive navigation control, top categories, payment distribution, warehouse health, and pipeline status. The page uses interactive filters and a responsive layout that adapts to the available screen width.

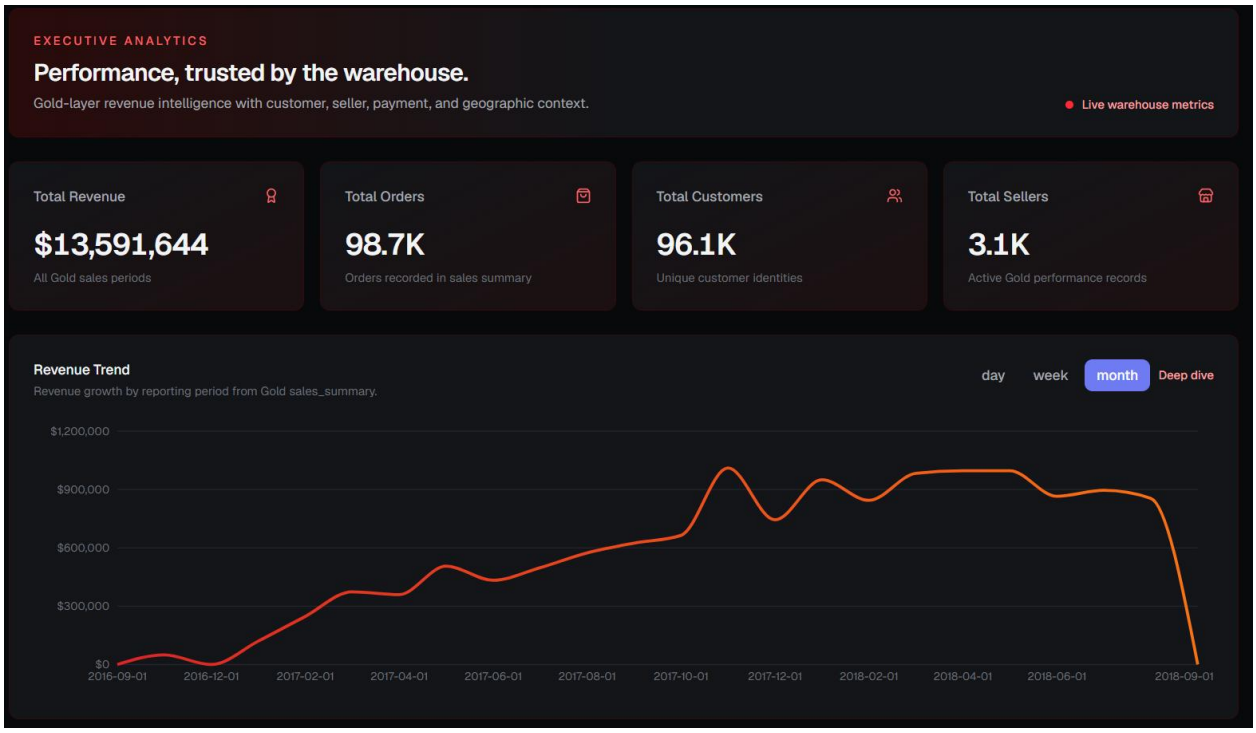


Figure 6. Executive Dashboard: KPI cards and revenue trend.

Revenue Intelligence

Revenue Intelligence explains revenue trend and revenue change analysis over time, revenue concentration and revenue drivers by category, category-level revenue distribution, top categories by contribution, and period-over-period analysis for identifying growth or decline.

Customer Intelligence

Customer Intelligence explains the customer funnel and journey visualization from acquisition through purchase, retention and drop-off analysis at each funnel stage, customer segmentation by value and behavior, and a customer overview summarizing the active customer base.

Seller Intelligence

Seller Intelligence explains the seller leaderboard ranked by revenue contribution, seller clustering by performance tier, revenue contribution by seller, seller distribution across the marketplace, and detailed seller performance analysis.

Category Intelligence

Category Intelligence explains category performance and a category treemap for visualizing relative scale, revenue concentration through a Pareto view, category rankings, and product contribution within each category.

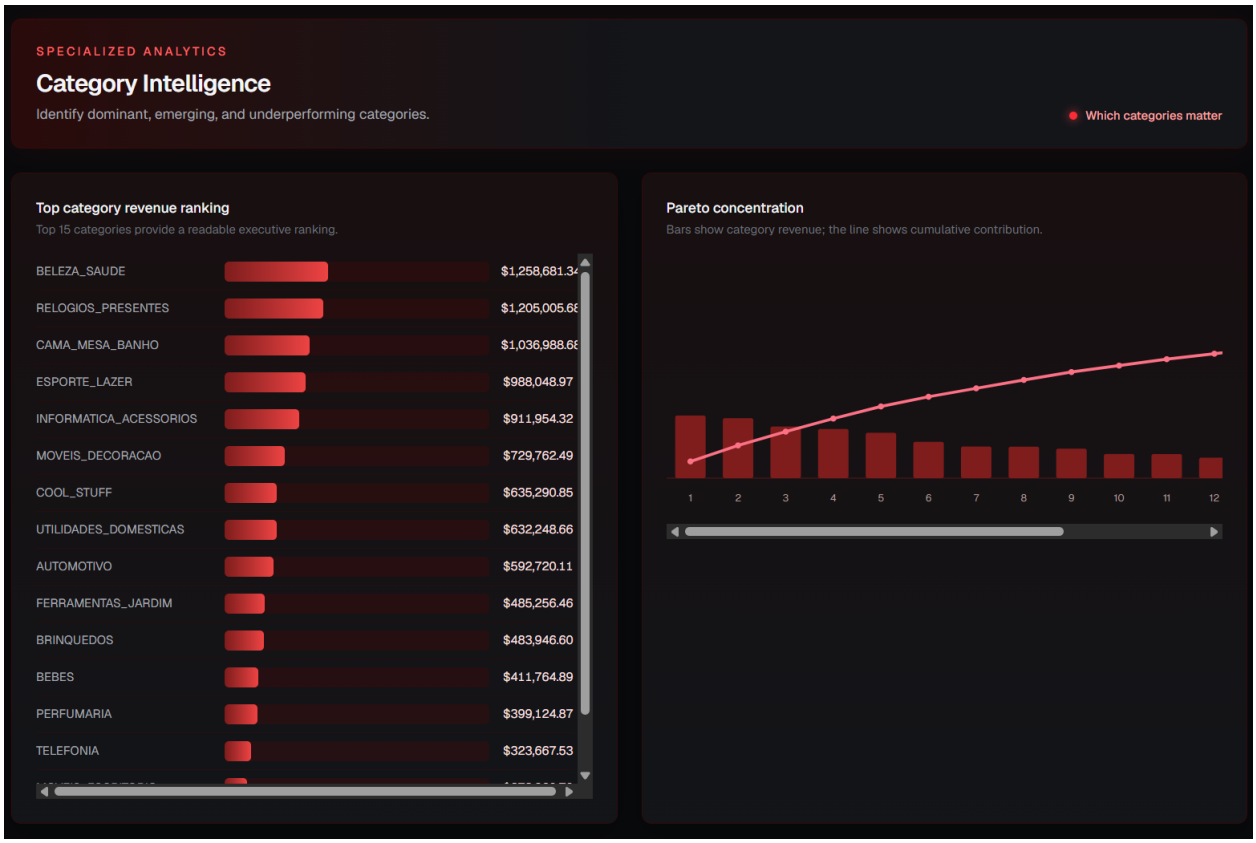


Figure 7. Category Intelligence: top category revenue ranking and Pareto concentration.

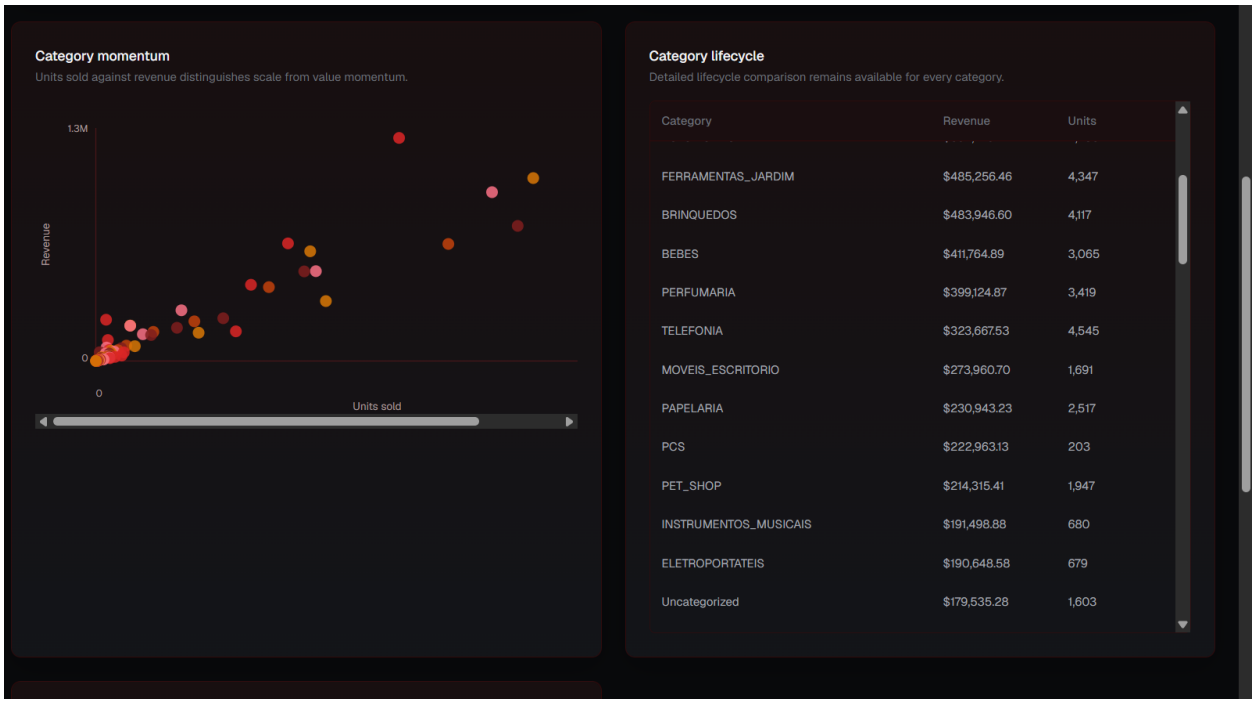


Figure 8. Category Intelligence: category momentum and lifecycle comparison.

Geographic Intelligence

Geographic Intelligence explains the regional opportunity map, revenue by state, regional analysis, geographical insights into customer and seller distribution, and opportunity visualization for underserved regions.

Data Quality Center

The Data Quality Center surfaces the complete Great Expectations posture: validation success rate, passed and failed expectation counts, total coverage, expectation families, audit status, a validation history trend, warehouse health, and observability signals for the most recent execution.

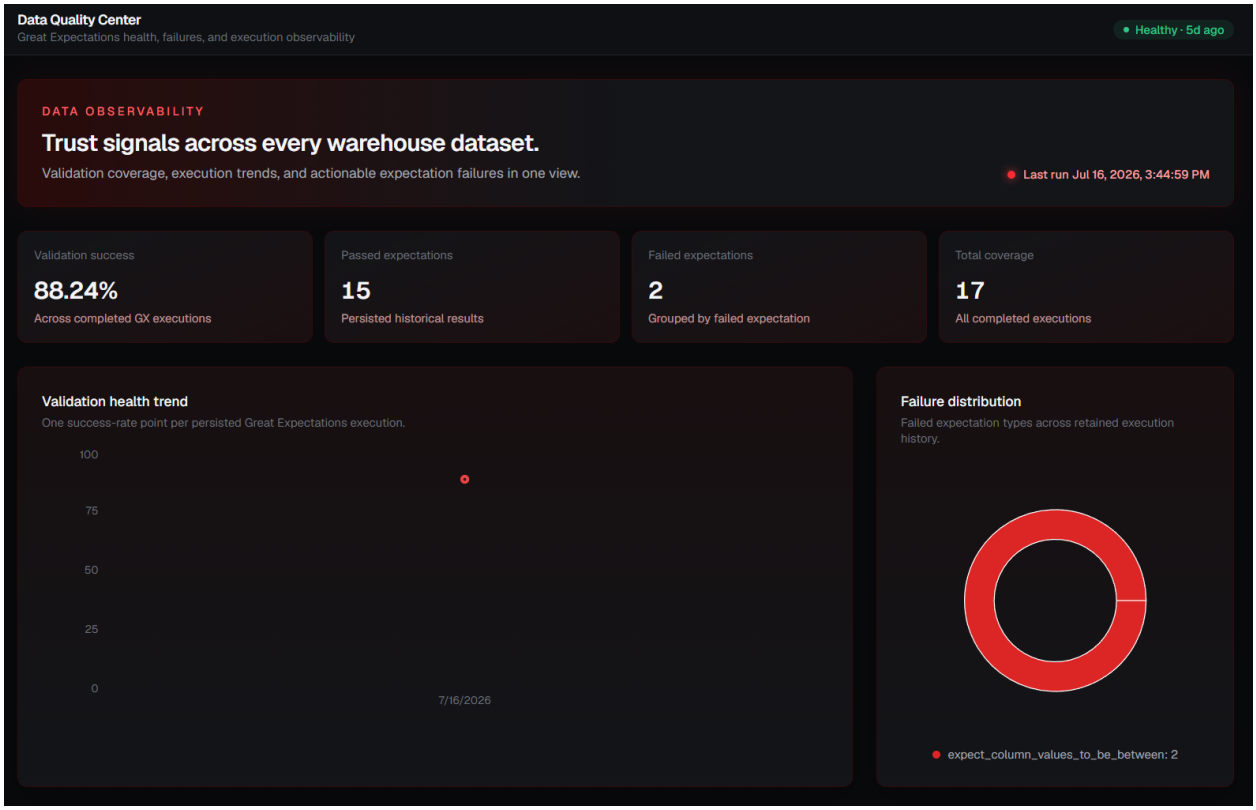


Figure 9. Data Quality Center: validation health trend and failure distribution.

Dataset Explorer

The Dataset Explorer walks the warehouse lineage from Bronze through Silver and Gold to the business transformations that make each layer ready for consumption. Each table card exposes row counts and a table detail panel describing the purpose of the table, supporting full table exploration and lineage tracing.

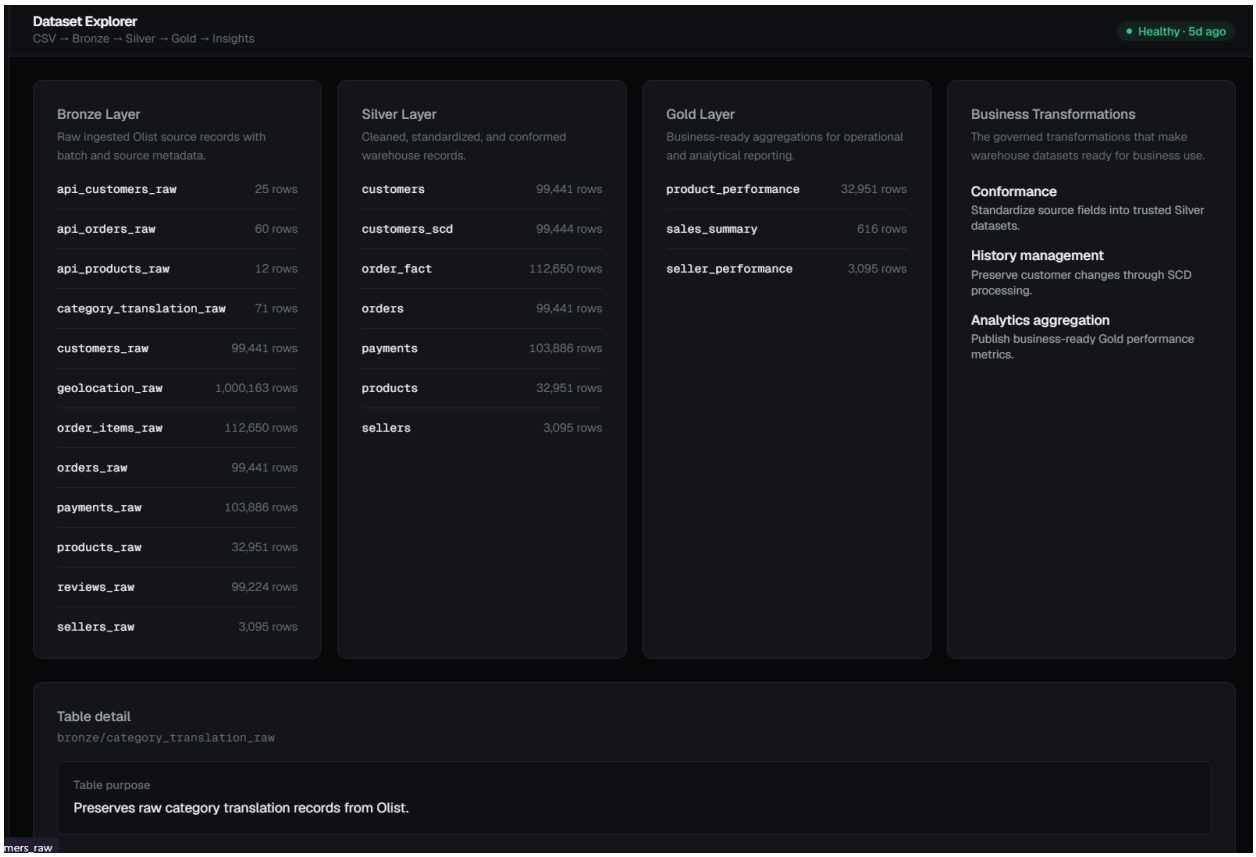


Figure 10. Dataset Explorer: Bronze, Silver, Gold, and Business Transformations.

Pipeline Health

Pipeline Health explains batch execution, pipeline monitoring, execution history, operational metrics, and warehouse status. It renders a pipeline execution timeline per stage and a DAG monitoring view reflecting the current warehouse dependency sequence and latest validation signal.

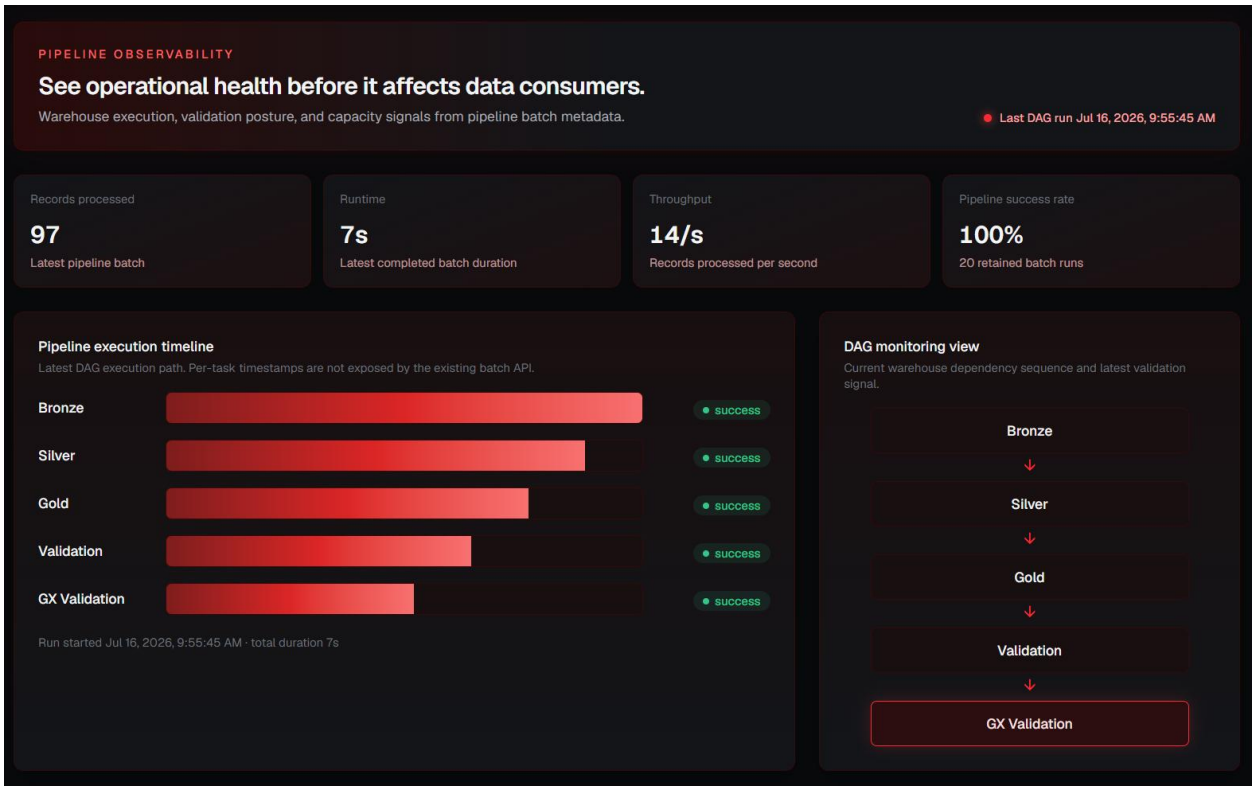


Figure 11. Pipeline Health: execution timeline and DAG monitoring view.

Responsive Dashboard

The dashboard is now fully responsive. Layouts are documented and tested for desktop, tablet, and mobile breakpoints, with adaptive chart resizing and responsive visual components that reflow KPI cards, tables, and charts to fit the available viewport without loss of information.

12. FastAPI Backend

The FastAPI backend has been expanded well beyond the original overview, pipeline, quality, medallion, analytics, and SCD endpoints. It now groups routes into REST APIs, Analytics APIs, Quality APIs, Pipeline APIs, SCD APIs, and Dashboard APIs, all returning a consistent data/meta response envelope that React Query consumes for caching, loading, empty, and error states.

API group	Representative responsibility
REST APIs	Overview and medallion endpoints describing warehouse structure and totals.
Analytics APIs	Revenue trend, top categories, seller performance, payment distribution, and customer funnel endpoints.
Quality APIs	Great Expectations quality score, validation history, and expectation results.
Pipeline APIs	Batch status, execution timeline, and DAG monitoring signals.
SCD APIs	Customer SCD summary and per-customer history.
Dashboard APIs	Aggregated endpoints tailored to each Business Intelligence dashboard page.

Every endpoint returns the same data/meta envelope shape, which lets the React Query layer apply a single set of hooks across the entire dashboard for caching, background refresh, and consistent loading, empty, and error UI states.

13. Great Expectations Framework

Great Expectations was chosen over ad hoc validation scripts because it provides declarative, versioned expectation suites, structured result objects, and a repeatable execution model that persists history rather than only printing pass/fail output to a console.

- Validation suites define expectations per warehouse table, covering nulls, uniqueness, referential integrity, volume, and positive-value ranges.
- Expectation execution runs the suite against the current warehouse state and produces a structured result for every expectation.
- Data quality reporting aggregates expectation results into passed/failed counts, a coverage total, and a quality score.
- Integration with the React dashboard renders validation health trend, failure distribution, and expectation family breakdowns on the Data Quality Center page.
- Integration with FastAPI exposes the latest and historical validation runs through the Quality APIs.
- Quality score generation converts the expectation pass rate for a run into the single score shown across the Executive Dashboard and Data Quality Center.

14. Observability Framework

Observability ties batch control, audit logs, validation results, and pipeline health into a single operational picture of the warehouse, so that data freshness and pipeline behavior are visible without querying the database directly.

Observability signal	Source
Batch control	metadata.batch_control status, timing, and processed/rejected record counts.
Audit logs	Persisted pipeline execution metadata across ingestion, transformation, and validation runs.
Validation	Great Expectations quality score, expectation results, and validation history.
Pipeline health	Execution timeline, throughput, and pipeline success rate across retained batch runs.
Warehouse monitoring	Bronze, Silver, and Gold row counts and freshness surfaced on the Dataset Explorer.
Data freshness	Last successful batch and last validation run timestamps.
Operational visibility	A single Pipeline Health and Data Quality Center view combining the signals above.

15. Startup Automation

A set of startup scripts automates local environment setup so the full stack can be brought up, checked, and torn down without manually sequencing each service.

Script	Purpose
start_project.bat	Starts Docker, PostgreSQL, the FastAPI backend, and the React dashboard in the correct order.
stop_project.bat	Stops all running project services and containers cleanly.
health_check.bat	Verifies that PostgreSQL, FastAPI, and the dashboard are reachable and reports service status.
launch_airflow.bat	Starts the Airflow webserver and scheduler and opens the warehouse_pipeline DAG view.

16. Sample Code Snippets

Bronze ingestion - create a batch record (scripts/ingestion/load_customers.py)

```
result = conn.execute(text("""
    INSERT INTO metadata.batch_control (pipeline_name, status)
    VALUES ('customers_ingestion', 'RUNNING')
    RETURNING batch_id
"""))
batch_id = result.scalar()
```

Silver transformation - standardize customer location (sql/silver/01_customers.sql)

```
SELECT DISTINCT ON (customer_id)
    customer_id, customer_unique_id,
    UPPER(customer_city), UPPER(customer_state), batch_id
FROM bronze.customers_raw
ORDER BY customer_id, batch_id DESC;
```

Gold aggregation - daily revenue (sql/gold/01_sales_summary.sql)

```
SELECT DATE(purchase_timestamp),
    COUNT(DISTINCT order_id), SUM(price), SUM(freight_value)
FROM silver.order_fact
GROUP BY DATE(purchase_timestamp);
```

SCD Type 2 - expire current row (scripts/scd.py)

```
UPDATE silver.customers_scd AS target
SET effective_end_date = CURRENT_TIMESTAMP, is_current = FALSE
FROM silver.customers AS source
WHERE target.customer_id = source.customer_id
AND target.is_current = TRUE
AND (target.customer_city IS DISTINCT FROM source.customer_city
OR target.customer_state IS DISTINCT FROM source.customer_state);
```

Great Expectations - table-level expectation suite (great_expectations/suites/silver_customers.py)

```
validator.expect_column_values_to_not_be_null('customer_id')
validator.expect_column_values_to_be_unique('customer_id')
validator.expect_column_values_to_be_between('payment_value', min_value=0)
validator.expect_table_row_count_to_be_between(min_value=1)
```

FastAPI endpoint - customer SCD history (backend/app/routes/scd.py)

```
@router.get('/history/{customer_id}')
def get_history(customer_id: str):
    history = get_customer_scd_history(customer_id)
    if history is None:
        raise HTTPException(status_code=404)
    return success_response(history.model_dump())
```

17. Challenges and Solutions

Challenge	Applied solution
PostgreSQL connectivity	Centralized connection helpers and Docker environment variables provide consistent access to de_poc.
Docker networking	Pipeline scripts use the de_poc_postgres container hostname within the Docker/Airflow environment.
Incremental updates	metadata.load_tracker records the last processed customer batch and limits the next load to newer Bronze rows.
Customer attribute changes	SCD Type 2 expires the existing current record and inserts a versioned replacement.
Validation coverage	Great Expectations suites replace ad hoc row-count scripts with declarative, repeatable expectations and a persisted quality score.

Challenge	Applied solution
Dashboard integration	React Query services consume FastAPI's consistent data/meta envelope and display loading, empty, and error states.
Responsive layout	Tailwind CSS breakpoints and adaptive chart sizing keep KPI cards, tables, and charts usable from desktop to mobile.

18. Outcomes

The proof of concept demonstrates a complete data engineering path from e-commerce CSV data to governed warehouse analytics and a presentation-ready, responsive dashboard.

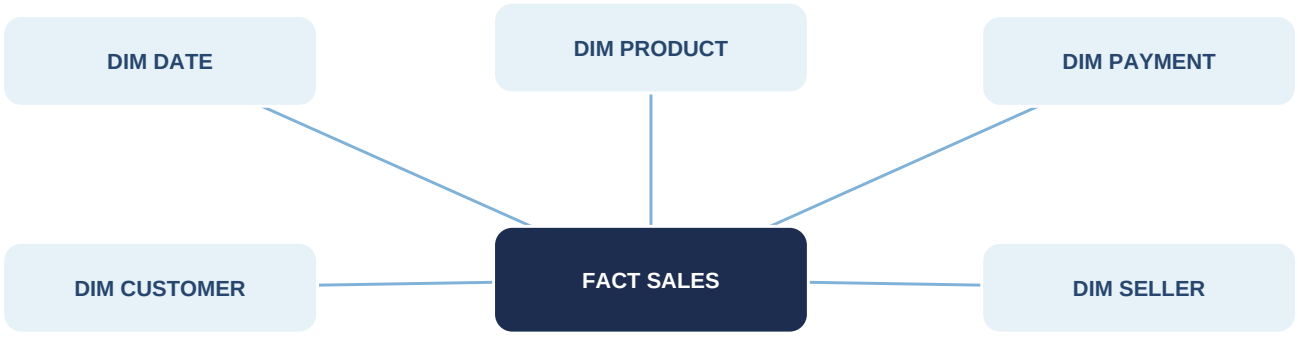
- Implemented: Bronze Layer
- Implemented: Silver Layer
- Implemented: Gold Layer
- Implemented: Incremental Loading
- Implemented: Metadata Framework
- Implemented: Audit Framework
- Implemented: Observability Framework
- Implemented: Validation Framework
- Implemented: Great Expectations Data Quality Suites
- Implemented: SCD Type 1
- Implemented: SCD Type 2
- Implemented: Airflow
- Implemented: FastAPI APIs
- Implemented: Business Intelligence Dashboard
- Implemented: Data Quality Center
- Implemented: Responsive Dashboard Design
- Implemented: Startup Automation Scripts

The resulting platform provides traceable ingestion, clean and reusable dimensional data, business metrics, observable pipeline status, declarative data quality validation, and a concrete demonstration of customer history management, all presented through a governed API and an executive-ready dashboard.

19. Power BI Analytics Add-on

Power BI report, semantic model, star schema, and executive dashboard evidence

The Power BI semantic model is built around the Gold analytical views in PostgreSQL.
The central fact table is gold.vw_powerbi_fact_sales, with dimension views for customer, date, product, seller, and payment. These relationships form a one-to-many star-schema model for report filtering, drill-down, and interactive analysis.
The model also includes dedicated SCD support views for customer history and Type 1 vs Type 2 demonstration, plus a forecast source used by the Forecast page.



19.2 Power BI Report - Sales Page

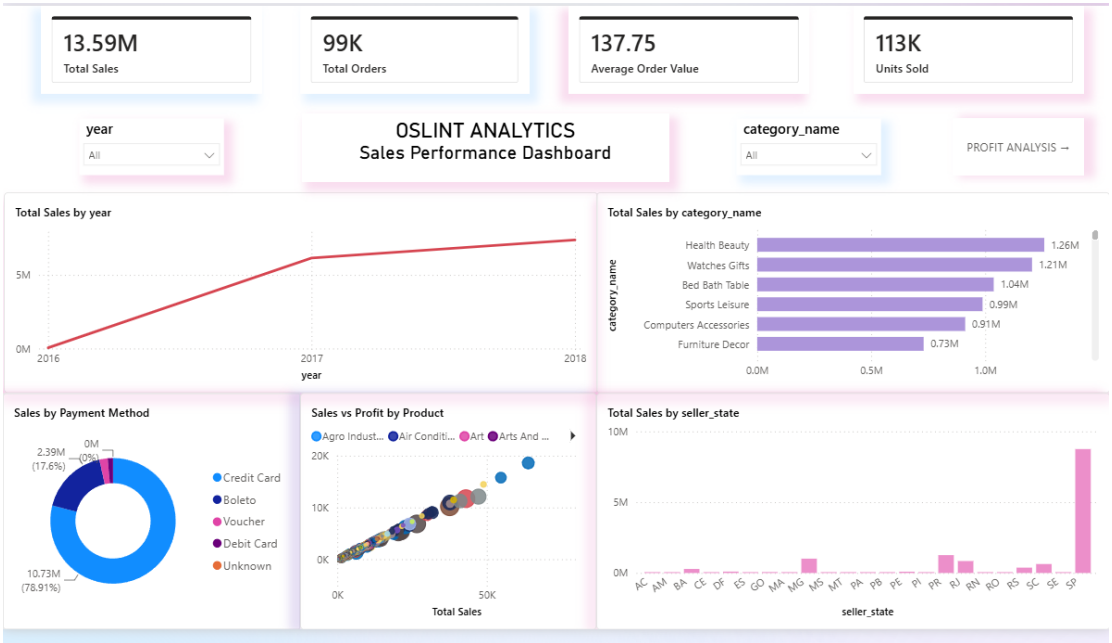


Figure 12. Power BI Sales dashboard with KPI cards, time trend, category analysis, payment mix, scatter analysis, and seller-state analys

19.3 Power BI Report - Profit and Forecast Pages

Profit Page

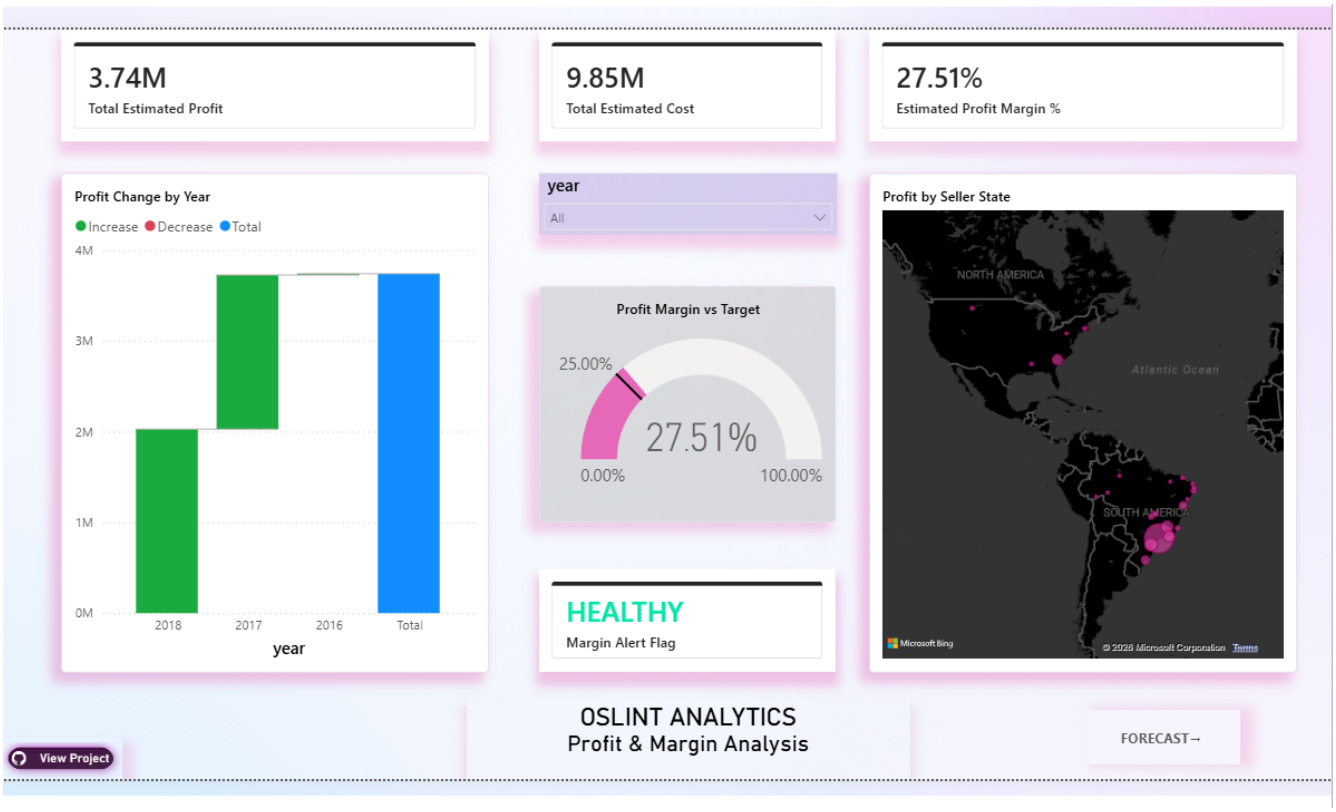


Figure 13. Profit page with estimated profit/cost/margin KPIs, waterfall analysis, margin target gauge, SCD-aware alert card, and geographic profit map.

Forecast Page

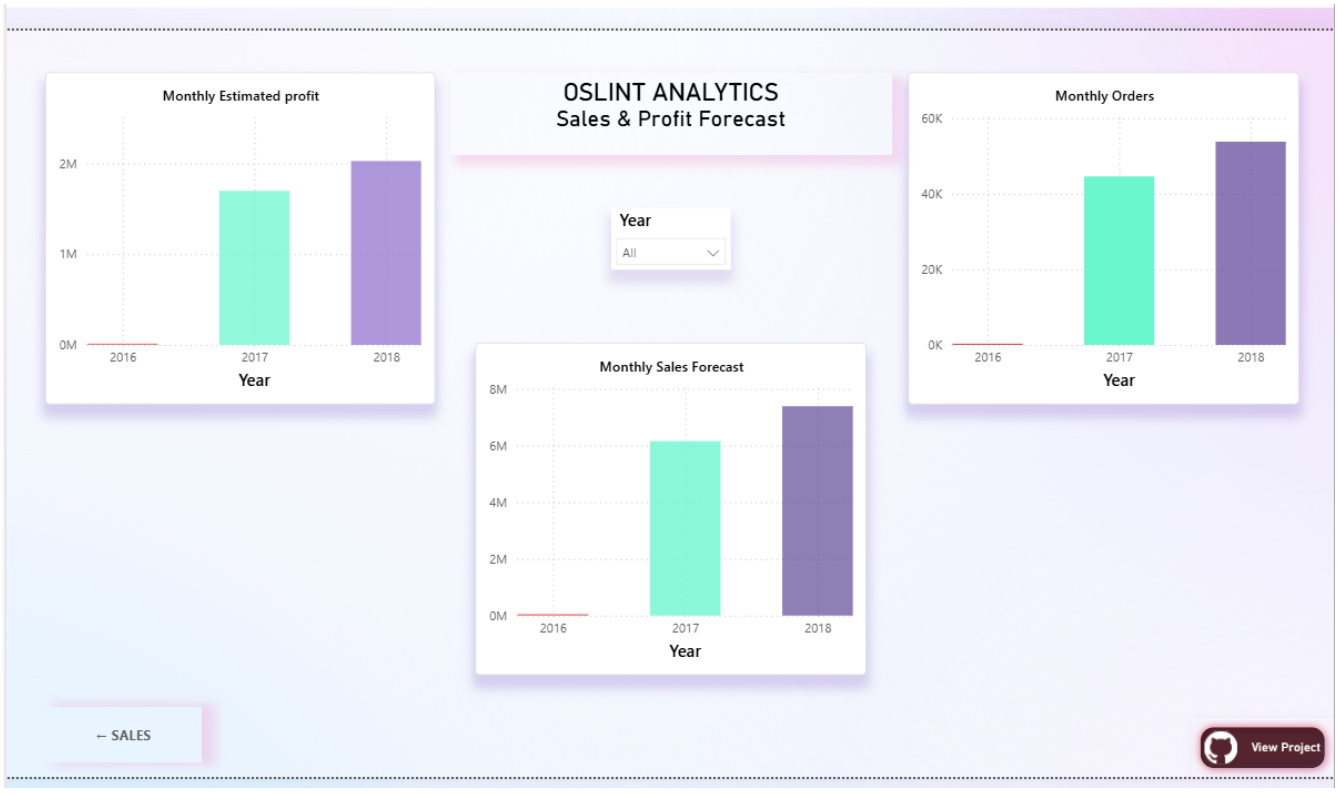


Figure 14. Forecast page with monthly sales, monthly estimated profit, monthly orders, year slicer, navigation, and project hyperlink.

19.4 Report Features Implemented

- Three analytical pages: Sales, Profit, and Forecast.
- KPI cards, line/bar/donut/scatter/column/waterfall/gauge/map visuals.
- Product Category slicer plus a synchronized Year slicer across pages.
- Drill paths for Date (Year > Quarter > Month > Day), Product (Category > Product), and Geography (State > City).
- Profit margin alerting with a 25% target gauge and dynamic status color.

19.5 Power BI Model View and SCD Demonstration

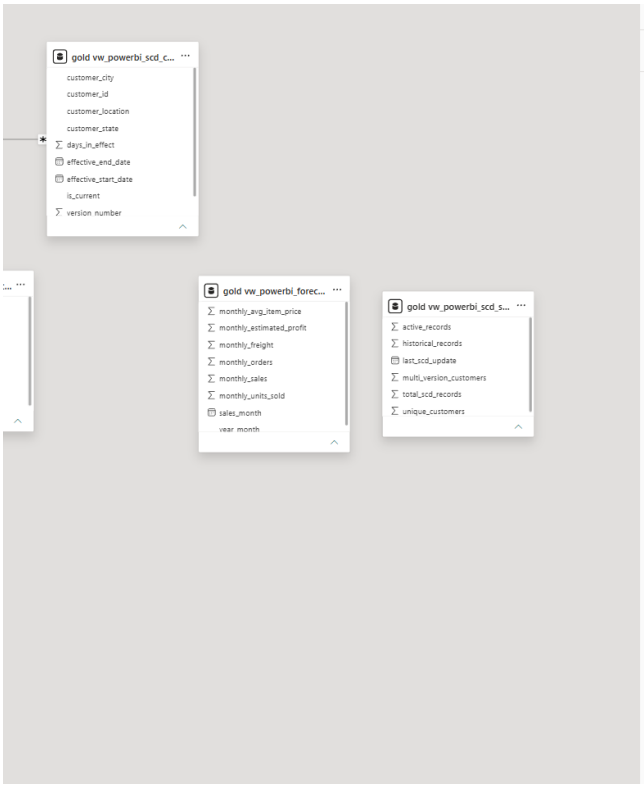


Figure 16. SCD support/forecast area of the Power BI model, including customer history, forecast source, and SCD summary views.

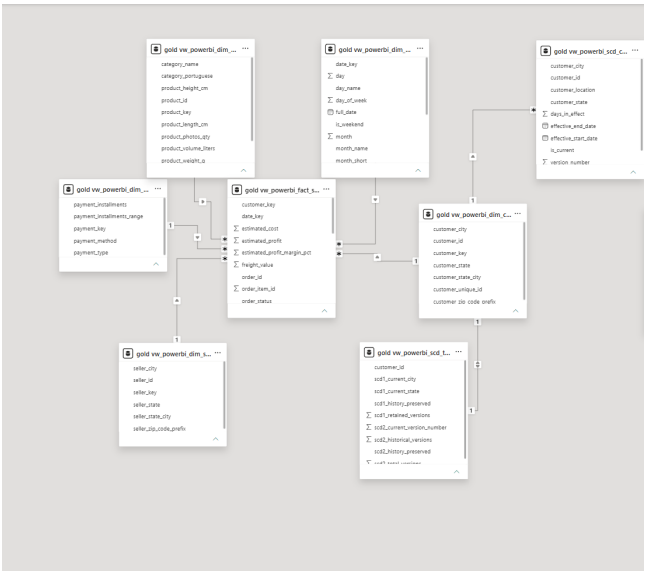


Figure 15. Power BI model view showing the star-schema relationships and connected SCD structures.